Threat Model: Login & Payment Flow

GRC Practicum — Project 3 | NOVAtech Inc. (simulated scenario) | Methodology: STRIDE

Note: NOVAtech Inc. is a fictional company; this threat model is simulated for demonstration. The methodology and document structure reflect real-world secure design review practice.

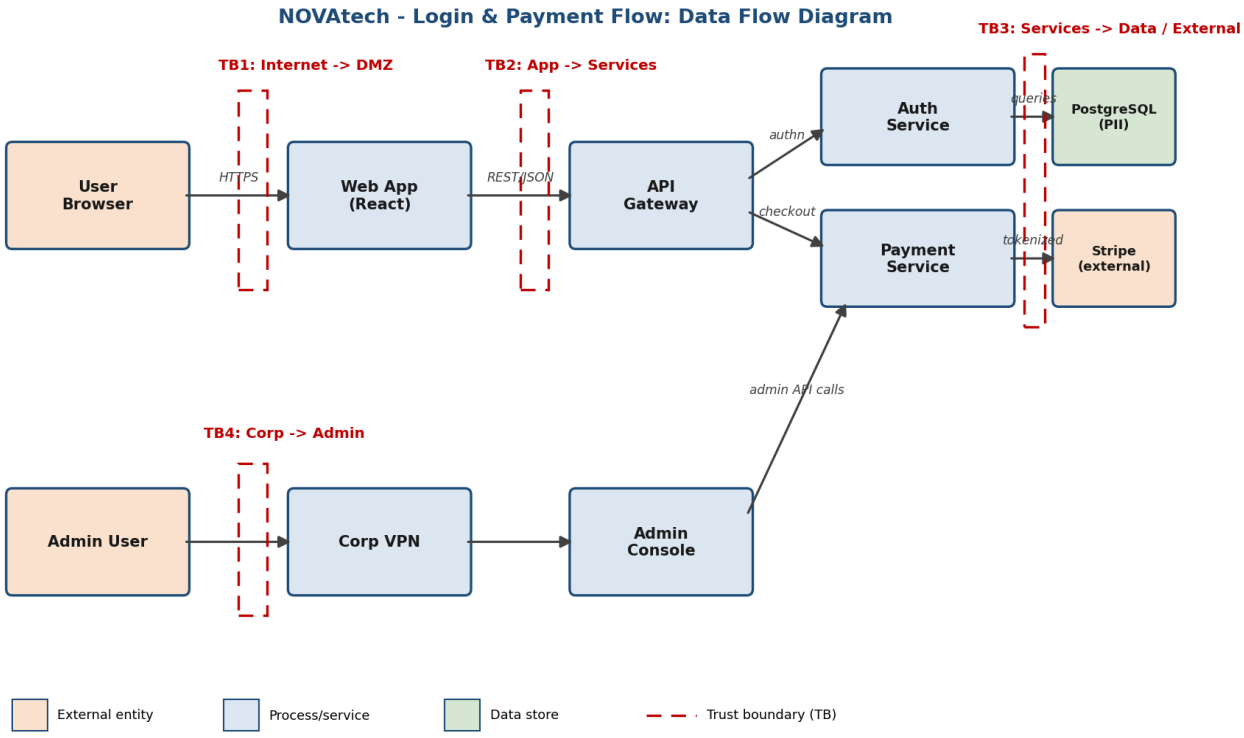
1. SCOPE & METHODOLOGY

This model covers NOVAtech's customer login and payment flow — the path crossing the most trust boundaries and touching the highest-value data (credentials, PII, payment) — plus the administrative access path. Out of scope: marketing site, internal HR systems, CI/CD pipeline (candidates for future models).

Method: (1) draw the data flow diagram and mark trust boundaries; (2) apply STRIDE per component — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; (3) for each credible threat, record the existing control, the gap, and a recommendation; (4) feed accepted gaps into the enterprise risk register.

2. SYSTEM DESCRIPTION & DATA FLOW DIAGRAM

A customer authenticates via browser to the React web app, which calls the API gateway; the gateway routes to the Auth service (sessions) and Payment service (checkout via Stripe tokenization). Customer PII resides in PostgreSQL. Administrators reach an admin console over the corporate VPN. Four trust boundaries (TB1–TB4) mark where control changes hands — threats concentrate at these crossings.



3. STRIDE ANALYSIS

STRIDE	Threat	Component (boundary)	Existing Control	Gap	Recommendation
Spoofing	Credential stuffing / password reuse	Auth Service (TB1)	Password policy only	No rate limiting; no MFA offered to customer accounts	Rate limiting + breached-pa + optional customer MFA
Spoofing	Session hijack via stolen token	Web App (TB1)	HTTPS in transit	Sessions not invalidated on password change	Rotate/invalidate all session tokens on credential chang
Tampering	Payment amount manipulation	API → Payment Service (TB2)	Client-side validation only	Server trusts client-supplied price field	Server-side validation; price database, never from client
Repudiation	User denies performing a transaction	Payment Service	Basic application logs	Logs lack integrity protection and user/IP binding	Structured audit logging to storage; NTP time sync
Info. disclosure	PII/card data written to logs	All services	None	Debug mode logs full request bodies	Log-scrubbing standard; rel tokenization exclusively
Info. disclosure	IDOR on /api/user/{id}	API Gateway (TB2)	Authentication only	No object-level authorization checks	Enforce per-object authoriz tests to code review checkli
Denial of service	Login endpoint flood	Auth Service (TB1)	CDN (basic tier)	No application-layer rate limiting	WAF rules with progressive authentication endpoints
Elev. of privilege	Admin console reachable from internet	Admin path (TB4)	VPN 'expected'	Console also exposed publicly (misconfiguration); no owner verifies	Restrict to VPN CIDR; MFA + on external access attempts
Elev. of privilege	Over-scoped payment API key	Payment Service (TB3)	Single Stripe key	Full-scope key shared across three services	Scoped restricted keys per s rotation standard

Severity note: gaps are prioritized qualitatively here; formal scoring occurs when each gap is transferred to the risk register (Project 2) using its defined likelihood/impact criteria.

4. FINDINGS → RISK REGISTER FEED

A threat model that does not update the register is a drawing, not a control. Disposition of the gaps above:

- **IDOR on user API (new risk R-16):** object-level authorization gap exposing customer PII; proposed inherent L3 × I4 = 12, owner: Eng Lead
- **Admin console exposure (new risk R-17):** internet-reachable privileged interface; proposed inherent L3 × I5 = 15, owner: Eng Lead — highest-priority finding of this model
- **Over-scoped payment API key (new risk R-18):** blast-radius amplifier for any service compromise; proposed inherent L2 × I4 = 8, owner: Cloud Lead
- **Reinforces existing R-02** (internet-facing exploitation): rate-limiting and WAF gaps feed its treatment plan
- **Payment tampering & logging gaps:** fold into secure-SDLC requirements (future policy: security requirements in development, ISO 27001 A.8.25–A.8.29)

5. THE STRUCTURE — REUSABLE SKELETON

What job each part of a threat model does (apply to any system):

- **Scope statement:** names what is modelled AND what is not. Unbounded models never finish; scope is what makes them repeatable per release.
- **DFD + trust boundaries:** the diagram is not decoration — boundaries are where assumptions change (internet→DMZ, app→data, corp→admin), and they tell you where to concentrate analysis.
- **Per-component STRIDE pass:** the checklist discipline. Each letter is a question asked of each component: can identity be faked here? data altered? actions denied? data leaked? service flooded? privileges escalated?
- **Existing control → Gap → Recommendation columns:** the same condition-vs-criteria thinking as an audit finding — what is, what should be, what to do. This is why threat modelling is a GRC skill, not just an engineering one.
- **Register feed:** every accepted gap becomes a scored, owned risk. This closing step connects design review to enterprise risk management — and it is the step most threat models skip.

The five translation moves from SOC experience: attacker TTPs you have watched in logs (MITRE ATT&CK) become hypothesized threats per component; detection gaps you have felt on shift become the 'Gap' column; and the escalation instinct becomes the register feed.